Chapter 07

Programming Principles

Exception Handling & File I/O

Mr. SOK Pongsametrey
metreysk@gmail.com

This week we'll connect error handling and file operations to Programming Principles like separation of concerns, maintainability, clarity, and robustness.

ABOUT

Why Exception Handling Matters

- Programs encounter unexpected situations (errors)
- Users expect applications to handle errors gracefully
- Proper error handling improves user experience and system reliability
- **Examples of exceptions:**
  - File not found
  - Network connection lost
  - Invalid user input
  - Division by zero

Principle tie-in: Robust error handling supports maintainability and reliability

What is an Exception?

- An event that disrupts normal program execution flow
- Java's way of handling runtime errors
- **Exception hierarchy:**
  - Throwable (top level)
  - Exception (checked exceptions)
  - RuntimeException (unchecked exceptions)
  - Error (system-level errors)

Principle tie-in: Structured error handling improves code clarity and debugging

Basic Try-Catch Structure

```
try {
    // Risky code that might throw an exception
    int result = 10 / 0;
    System.out.println("Result: " + result);
} catch (ArithmeticException e) {
    // Handle specific exception
    System.out.println("Error: Cannot divide by zero!");
    System.out.println("Details: " + e.getMessage());
} catch (Exception e) {
    // Handle any other exception
    System.out.println("Unexpected error: " + e.getMessage());
}
```

- **Principle tie-in: Separation of concerns - normal logic separated from error handling**

The Finally Block

```
FileReader file = null;
try {
    file = new FileReader("data.txt");
    // Process file
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} finally {
    // Always executes - cleanup code
    if (file != null) {
        try {
            file.close();
        } catch (IOException e) {
            System.out.println("Error closing file");
        }
    }
    System.out.println("Cleanup completed");
}
```

- **Finally block always executes** (success or failure)
- **Used for resource cleanup** (files, connections, etc.)
- **Principle tie-in: Ensures reliable resource management**

Try-with-Resources (Modern Approach)

```
// Automatic resource management
try (FileReader file = new FileReader("data.txt");
     BufferedReader reader = new BufferedReader(file)) {

    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
// Resources automatically closed
```

- **Automatic resource closure** when try block ends
- **Cleaner, more reliable code**
- **Principle tie-in: KISS principle - simpler syntax, fewer bugs**

Common Exception Types

Exception Type	When it occurs	Example
NullPointerException	Accessing null object	String s = null; s.length();
ArrayIndexOutOfBoundsException	Invalid array index	arr[10] when array has 5 elements
FileNotFoundException	File doesn't exist	Opening non-existent file
IOException	Input/output problems	Network interruption
NumberFormatException	Invalid number conversion	Integer.parseInt("abc")

- **Principle tie-in: Understanding exception types helps write more reliable code**

Custom Exceptions

```
// Create custom exception class
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

// Using custom exception
public void validateAge(int age) throws InvalidAgeException {
    if (age < 0 || age > 150) {
        throw new InvalidAgeException("Age must be between 0 and 150, got: " +
age);
    }
}

// Calling method with custom exception
try {
    validateAge(-5);
} catch (InvalidAgeException e) {
    System.out.println("Invalid age: " + e.getMessage());
}
```

- **Principle tie-in: Create domain-specific exceptions for better error communication**

File I/O (Input/Output) is essential in Programming Principles for several interconnected reasons that directly support core development practices and real-world application needs:

FILE I/O

File I/O Fundamentals

- **File operations are essential** for data persistence
- **Common file operations:**
  - Reading text files
  - Writing text files
  - Processing CSV data
  - Handling binary files
- **Java file I/O classes:**
  - File - represents file/directory
  - FileReader/FileWriter - character streams
  - BufferedReader/BufferedWriter - buffered I/O
  - Scanner - convenient reading

- **Principle tie-in: File I/O enables separation between data and logic**

Reading Files - Multiple Approaches

- **Approach 1: BufferedReader (Recommended)**

```
try (BufferedReader reader = new BufferedReader(new
FileReader("students.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println("Student: " + line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
```

- **Approach 2: Scanner**

```
try (Scanner scanner = new Scanner(new File("numbers.txt"))) {
    while (scanner.hasNextInt()) {
        int number = scanner.nextInt();
        System.out.println("Number: " + number);
    }
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
}
```

Principle tie-in: Multiple approaches support different use cases (flexibility)

Writing Files

- **Writing Text Files:**

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt")))
{
    writer.write("Hello, File I/O!");
    writer.newLine();
    writer.write("Programming Principles in Java");
    writer.newLine();
    writer.write("Exception Handling and File Operations");
} catch (IOException e) {
    System.out.println("Error writing file: " + e.getMessage());
}
```

- **Appending to Files:**

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("log.txt",
true))) {
    writer.write(new Date() + ": Application started");
    writer.newLine();
} catch (IOException e) {
    System.out.println("Error writing to log: " + e.getMessage());
}
```

Principle tie-in: Consistent file handling patterns improve maintainability

Real-world AI Example - Log Management

```
public class AIConversationLogger {
    private String logFile;

    public AIConversationLogger(String logFile) {
        this.logFile = logFile;
    }

    public void logInteraction(String userInput, String aiResponse) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(logFile, true))) {
            String timestamp = new Date().toString();
            writer.write(timestamp + " | USER: " + userInput);
            writer.newLine();
            writer.write(timestamp + " | AI: " + aiResponse);
            writer.newLine();
            writer.write("---");
            writer.newLine();
        } catch (IOException e) {
            System.err.println("Failed to log conversation: " +
e.getMessage());
            // Don't crash the application if logging fails
        }
    }
}
```

Principle tie-in: Robust error handling ensures AI applications continue running even when logging fails

Best Practices for Exception Handling

✅ Do's:

- **Catch specific exceptions** first, general ones last
- **Use try-with-resources** for automatic cleanup
- **Log exceptions** with meaningful messages
- **Provide user-friendly error messages**
- **Clean up resources** in finally blocks
- **Validate input** before processing

❌ Don'ts:

- **Don't catch and ignore** exceptions
- **Don't use exceptions for control flow**
- **Don't expose sensitive information** in error messages
- **Don't catch Exception unless necessary**

Principle tie-in: Following best practices improves maintainability and security

Lab Activity - Student Grade System

- **Build a robust student grade management system**
  - Read student grades from CSV file (students.csv)
  - Calculate average grade for each student
  - Write results to output file (results.txt)
  - Handle all possible exceptions gracefully
  - Create log file for all operations

CSV Format:

```
Name,Math,Science,English
Alice,85,92,78
Bob,79,88,91
Carol,95,87,89
```

- **Focus: Practice separation of concerns, reliability, and maintainability**

Exercise: Secure Login System

Build a file-based authentication system

Requirements:

- **User data storage:** Read username/password from users.txt
- **Login validation:** Check credentials against file data
- **Session logging:** Log all login attempts (success/failure)
- **Error handling:** Handle missing files, invalid input, file corruption
- **Security:** Don't expose passwords in error messages

Reflection Questions:

- How does proper exception handling improve user experience?
- Which Programming Principles did you apply?
- How would you extend this system for a real application?

Wrap-up and Q&A

- Try-Catch isolates error handling
- finally ensures cleanup
- File I/O enables persistent data storage
- Next Week: Algorithms and Problem Solving